A

Project Report

on

A Comparative Study of Managed Free-Tier SQL Databases Under E-Commerce Workloads: Supabase vs CockroachDB vs Neon

submitted as partial fulfilment for the award of

BACHELOR OF TECHNOLOGY

SESSION 2025-26

in

Computer Science & Engineering (AI/AI&ML)

By

Avyaansh Dixit (2200291520055)

Under the supervision of

Ms. Nidhi Singh & Ms. Deepinder Kaur

KIET Group of Institutions, Ghaziabad

Affiliated to

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

(Formerly UPTU)

May, 2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Name: Avyaansh Dixit

Roll No: 2200291520055

Date: 26-03-2026

CERTIFICATE

This is to certify that Project Report entitled **Performance Evaluation of Free-Tier Cloud Databases Using an E-Commerce Workload** which is submitted by **Avyaansh Dixit** in partial fulfilment of the requirement for the award of degree B. Tech. in Department of Computer Science & Engineering (AI/AI&ML) of Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a record of the candidates own work carried out by them under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Ms. Nidhi Singh
Assistant Professor, CSE(AI&ML)
&
Ms. Deepinder Kaur (Co-Guide)
Assistant Professor, CSE(AI)

Dr. Rekha Kashyap
Dean-CSE(AI/AI&ML)

Date: 23-04-2026

ACKNOWLEDGEMENT

It gives us a great sense of pleasure to present the report of the B. Tech Project undertaken during B. Tech. Final Year. We owe special debt of gratitude to Ms. Nidhi Singh, Assistant Professor, Department of Computer Science & Engineering (AI&ML), KIET Group of Institutions, Ghaziabad and Ms. Deepinder Kaur, Assistant Professor, Department of Computer Science & Engineering (AI) for their constant support and guidance throughout the course of our work. Their sincerity, thoroughness and perseverance have been a constant source of inspiration for us. It is only their cognizant efforts that our endeavours have seen light of the day.

We also take the opportunity to acknowledge the contribution of Dr. Rekha Kashyap, Dean, Department of Computer Science & Engineering (AI&ML), KIET Group of Institutions, Ghaziabad, for her full support and assistance during the development of the project. We also do not like to miss the opportunity to acknowledge the contribution of all the faculty members of the department for their kind assistance and cooperation during the development of our project. Last but not the least, we acknowledge our friends for their contribution in the completion of the project.

Date: 23-04-2026

Name: Avyaansh Dixit (2200291520055)

ABSTRACT

This project presents a practical performance evaluation of three cloud-hosted SQL databases—Supabase, Neon, and CockroachDB—using a realistic e-commerce workload. The study focuses on understanding how these databases behave under common online shopping operations such as search, browsing, cart updates, and checkout. A structured six-table schema was designed to represent core e-commerce entities, and a synthetic dataset was generated using Python-based tools to simulate real-world data patterns.

The workload was designed with a read-heavy distribution (93% read and 7% write operations) to reflect actual user behaviour in e-commerce systems. Each database was tested using identical schema, dataset, queries, and execution conditions to ensure fairness. Performance was measured using key metrics including average latency, P95 and P99 latency, throughput, and error rate.

The results show clear differences in how the databases handle both read-heavy and transaction-heavy operations. CockroachDB demonstrated the best overall performance, particularly in terms of latency and throughput, while Supabase and Neon showed comparatively higher response times, especially for complex transactional operations such as checkout.

This project highlights the importance of workload-aware database selection and provides a simple, reproducible benchmarking approach that can help developers and startups make informed decisions when choosing a database for e-commerce applications.

TABLE OF CONTENTS

Title	Page
DECLARATION	ii
CERTIFICATE	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
LIST OF FIGURES	viii
LIST OF TABLES	ix
CHAPTER 1 (INTRODUCTION)	1
1.1 Introduction	1
1.2 Project Description	3
CHAPTER 2 (RELATED WORK)	5
2.1 Benchmark Methodology and Evaluation Frameworks	5
2.2 SQL and Relational Database Performance	6
2.3 Cloud-Hosted and Distributed SQL Systems	6
2.4 Workload Design for E-Commerce and User Behaviour	7
2.5 Positioning of Current Study	7
CHAPTER 3 (METHODOLOGY)	10
3.1 Overview	10

3.2 Database Selection	10
3.3 Schema Design	11
3.4 Dataset Design and Size	11
3.5 Data Generation and Loading	14
3.6 Indexing Strategy	14
3.7 Workload Design	15
3.8 Query Structure	17
3.9 Execution Strategy	18
3.10 Performance Metrics	20
3.11 Summary of the Methodology	21
CHAPTER 4 (RESULTS)	22
4.1 Supabase Results	24
4.2 CockroachDB Results	24
4.3 Neon Results	26
4.4 Cross-Comparison of the Three Databases	28
4.5 Short Concluding Summary for the Results Section	29
CHAPTER 5 (FUTURE SCOPE)	30
REFERENCES	32
PROOF OF SUBMISSION	35
RESEARCH PAPER/PATENT	36
PLAGIARISM REPORT	54

List of figures

Figure No.	Figure Name	Page
Fig. 1	Operation-wise Average Latency for Supabase	23
Fig. 2	Operation-wise Average Latency for CockroachDB	25
Fig. 3	Operation-wise Average Latency for Neon	27

List of tables

Table No.	Table Name	Page
Table 1	Source Summary Table	9
Table 2	E-commerce Schema and Dataset Specification	12
Table 3	Indexing Strategy and Purpose	15
Table 4	Workload Tier Distribution	15
Table 5	Operation-wise Workload Breakdown (10,000 Queries)	16
Table 6	Performance Metrics Definition and Collection Method	20
Table 7	Supabase — averaged results across 3 iterations	22
Table 8	CockroachDB — averaged results across 3 iterations	24
Table 9	Neon — averaged results across 3 iterations	26
Table 10	Final combined summary across all three databases	28

CHAPTER 1
INTRODUCTION

1.1 Introduction

India provides a timely setting for studying early-stage e-commerce infrastructure choices because both online commerce and startup formation are expanding rapidly. The Indian e-commerce market is projected to grow from US$125 billion in 2024 to US$345 billion by 2030, with longer-term projections reaching US$550 billion by 2035; this growth is linked to increasing adoption beyond major metros, especially in tier-2 and tier-3 cities. In parallel, the online shopper base increased from 140 million in 2020 to nearly 260 million in 2024, with projections of 300 million by 2030 and 700 million by 2035 [1], again with significant contributions from tier-2 and tier-3 cities. This market expansion intersects directly with the startup economy: India has been described as the world's third-largest startup ecosystem, with over 1.57 lakh DPIIT-issued startup recognition certificates as of December 31, 2024, and an ecosystem supported by more than 100 unicorns. Startup activity is not limited to large hubs such as Bengaluru, Hyderabad, Mumbai, and Delhi-NCR, as more than 51% of startups are reported to be emerging from tier II/III cities [2]. Consistent with this momentum, India's e-commerce sector also became a major driver of startup investment in 2024–25, attracting US$3.1 billion across 79 deals (31% of total startup funding), representing a sharp increase from 2023. Together, these trends make database infrastructure decisions for early e-commerce startups in India a practical and economically relevant research problem.

This paper focuses on cloud-hosted database services commonly positioned for modern application development. Supabase is presented in its official

documentation as a Postgres development platform in which every project begins with a full PostgreSQL database and includes integrated capabilities such as authentication and instant APIs [3]; importantly for startups, it is offered as a hosted platform designed to reduce infrastructure management effort and includes a free tier with flexible pricing for scaling. CockroachDB Serverless is introduced from official documentation as a distributed SQL database for cloud-native and globally distributed applications; it is fully managed, automatically scales compute and storage, requires no server configuration, and is PostgreSQL wire-protocol compatible, supporting standard SQL and ACID-compliant transactions [5], with a free-tier serverless entry point. For contextual contrast against non-relational approaches used in industry, MongoDB Atlas is described in official materials as a fully managed, multi-cloud database service built on the document model, designed to simplify deployment and operations (including provisioning, backups, and scaling) and supporting deployment on major cloud providers [4].

Managed, cloud-hosted databases are especially relevant for early-stage startups because they reduce the need for dedicated infrastructure work while enabling teams to deploy production-ready data layers quickly. Supabase explicitly emphasizes simplicity of getting started without managing infrastructure, while MongoDB Atlas highlights automation of operational tasks such as provisioning, backups, scaling, and performance optimization; Cockroach DB Serverless similarly emphasizes automatic scaling and the absence of infrastructure configuration. These properties align with the constraints typical of early product teams that need to validate demand and iterate rapidly [6].

The primary emphasis of this study is on SQL systems because early e-commerce workloads are fundamentally transactional and relational. Comparative research on SQL versus NoSQL in e-commerce contexts notes

that SQL databases provide strong consistency and ACID compliance, which supports transactional systems and complex relationships, while NoSQL systems can offer advantages in scalability and speed for large-scale applications [7]. Practical engineering guidance similarly highlights that e-commerce platforms involve relational data spanning sales, inventory, and customer management, and that SQL databases are well-suited when complex transactions, data integrity, and structured relationships must be maintained consistently across related entities. These characteristics motivate an SQL-centered evaluation for common e-commerce operations.

Accordingly, this paper compares cloud-hosted SQL databases under a shared early e-commerce workload using a controlled and reproducible evaluation setup. Each system is tested using the same schema, the same dataset, the same query mix, and the same measurement procedure to support fair comparison. The workload is designed to reflect realistic e-commerce operations including search and browsing, product detail access, cart updates, checkout, and payment completion, enabling observation of differences in latency, throughput, and operational behaviour in a startup-oriented setting. The paper does not propose a new benchmark standard; instead, it applies a consistent evaluation approach to compare selected services in a practical manner. The overall motivation is to help early-stage e-commerce startups make a more informed and pragmatic database choice.

The rest of the paper is organized as follows: Section 2 presents the Literature Review; Section 3 describes the Methodology; Section 4 reports the Results; Section 5 concludes the paper; Section 6 outlines Future Scope; and Section 7 lists References.

1.2 Project Description

This Research Project presents a simple and repeatable way to compare popular cloud-based database systems. It uses practical e-commerce examples

that reflect the real needs of growing online businesses. The study tests different types of workloads, including read, write, and mixed operations, while also applying basic techniques for improving queries and indexing. The focus is on measuring performance and reliability in a clear and transparent way, making it easy to understand and compare results. The findings show that free-tier cloud databases handle workloads differently, with each having its own strengths and limitations. Overall, this work helps users make better, more informed decisions when choosing a database, instead of relying on trial and error.

CHAPTER 2
LITERATURE REVIEW

Performance evaluation of database systems has evolved significantly with the emergence of cloud-hosted solutions and specialized workload requirements. Evaluating cloud-hosted SQL databases requires careful consideration of benchmark methodology, workload design, and infrastructure constraints. This review synthesizes prior work on database performance evaluation, cloud SQL systems, and e-commerce workload characteristics to position our comparative study.

2.1 Benchmark Methodology and Evaluation Frameworks.

Rigorous database evaluation requires standardized measurement approaches that enable fair comparison across systems. Li and Manoharan developed a key-value abstraction layer to compare SQL and NoSQL databases using fundamental operations—instantiation, read, write, delete, and key iteration—executed repeatedly with averaged timing. Their methodology demonstrates that performance varies substantially by operation type, with some systems optimized for writes while others excel at reads [8]. Cooper et al. introduced the Yahoo! Cloud Serving Benchmark (YCSB), which has become a de facto standard for evaluating NoSQL systems through configurable workload definitions and multi-phase testing. YCSB supports coordination of multiple clients to simulate realistic concurrent load, providing throughput and latency measurements under varying conditions. While YCSB was designed primarily for NoSQL systems, its methodology of workload-driven evaluation and multi-client stress testing provides valuable guidance for SQL database benchmarking [9].

2.2 SQL and Relational Database Performance.

Several studies have examined SQL database performance in contexts relevant to our work. Van der Veen et al. compared PostgreSQL and SQLite alongside time-series databases for sensor data storage on edge devices, finding that PostgreSQL delivered competitive or superior performance for typical sensor workloads on resource-constrained single-board computers. This work demonstrates that traditional SQL systems can perform well in edge computing scenarios when properly configured [10]. Pungila et al. evaluated multiple database systems including PostgreSQL, MySQL, and SQLite for smart meter data storage, finding that performance trade-offs varied significantly based on read/write balance and data volume. These studies suggest that SQL databases remain viable for specialized workloads when the data model and query patterns align with relational strengths [11].

2.3 Cloud-Hosted and Distributed SQL Systems.

The emergence of managed SQL services and distributed SQL architectures has introduced new performance considerations. Kiefer compared PostgreSQL with TimescaleDB, a time-series extension of PostgreSQL, finding that TimescaleDB achieved up to 20× higher insert rates for large-scale time-series data while delivering significantly faster time-based queries. This demonstrates that specialized SQL extensions can substantially improve performance for specific use cases [12]. Makris et al. evaluated PostgreSQL in cloud environments, noting that spatial indexing and query optimization are critical factors for geospatial workloads. Klein et al [13]. presented a healthcare system evaluation comparing three NoSQL databases, noting that systematic prototyping with application-specific workloads is essential because general benchmarks may not reflect real-world performance [14]. Their methodology of defining representative use cases and measuring

performance under realistic conditions informs our approach to e-commerce workload design.

2.4 Workload Design for E-Commerce and User Behaviour.

Understanding e-commerce user behaviour is essential for designing representative database workloads. The TPC-W benchmark, designed specifically for e-commerce systems, provides standardized workload definitions that simulate browsing, shopping, and ordering activities [15]. However, TPC-W was developed for traditional client-server architectures and may not fully capture modern cloud-based e-commerce patterns [16]. Research on consumer search behaviour indicates that users spend varying amounts of time examining product information, with search and filtering operations being among the most frequent database interactions in online retail environments. Studies of shopping cart and checkout processes reveal that these operations, while less frequent than browsing, are critical for transactional integrity and must be evaluated under appropriate consistency models [17]. The operational profile of an e-commerce system typically exhibits an 80/20 read/write ratio, with reads dominated by product searches and detail views, while writes are concentrated in cart updates and order completion. These behavioural patterns inform our benchmark workload design, emphasizing realistic operation mixes rather than synthetic uniform distributions [18].

2.5 Positioning of Current Study.

While extensive research has evaluated NoSQL databases and compared SQL versus NoSQL performance, limited work has systematically compared cloud-hosted SQL database options in the context of early-stage e-commerce applications [19]. Existing comparisons often focus on large-scale cloud deployments or compare SQL against NoSQL rather than evaluating multiple

SQL implementations [20]. Furthermore, most studies test databases in high-resource cloud environments, while edge computing scenarios with constrained resources remain underexplored [21]. Our study addresses these gaps by evaluating cloud-hosted SQL databases—including managed PostgreSQL (Supabase), distributed SQL (CockroachDB Serverless), and traditional relational systems—under e-commerce workloads representative of early-stage startup requirements. By testing on resource-constrained edge devices, we provide practical guidance for database selection in scenarios where infrastructure costs and local data processing capabilities are primary concerns [22].

Table 1: Source Summary Table

#	Title	Type	Contribution to Study	Relevance
RW1	A performance comparison of SQL and NoSQL databases (Li & Manoharan, 2013)	Paper	Demonstrates controlled, operation-level evaluation methodology; shows performance varies by operation type	Provides benchmark method for CRUD operations
RW2	Benchmarking cloud serving systems with YCSB (Cooper et al., 2010)	Paper	Introduces YCSB framework for workload-driven evaluation; multi-client stress testing	Standard methodology for database benchmarking
RW3	Sensor Data Storage Performance: SQL or NoSQL, Physical or Virtual (van der Veen et al., 2012)	Paper	Evaluates PostgreSQL and SQLite on edge devices; compares physical vs. virtual performance	Edge computing SQL evaluation precedent
RW4	Benchmarking database systems for sensor readings (Pungila et al., 2009)	Paper	Compares PostgreSQL, MySQL, SQLite for sensor data; read/write balance analysis	SQL performance for time-series workloads
RW5	Timescale DB vs. PostgreSQL for time-series (Kiefer, blog)	Blog	Performance comparison showing 20× insert improvement with Timescale DB extension	PostgreSQL extensions and optimization
RW6	Performance Evaluation of MongoDB and PostgreSQL for spatio-temporal data (Makris et al., 2019)	Paper	Cloud PostgreSQL evaluation; spatial indexing impact	Cloud SQL query optimization
RW7	Performance Evaluation of NoSQL Databases: A Case Study (Klein et al., 2015)	Paper	Healthcare system evaluation method; application-specific workload design	Use-case driven benchmarking approach
RW8	TPC-W E-Commerce Benchmark Evaluation (TPC, 2002-2003)	Standard	Standardized e-commerce workload definition; browsing/shopping/ordering mix	E-commerce benchmark reference
RW9	2024 E-Commerce Checkout: Baymard Research (Baymard Institute, 2024)	Report	Current checkout UX patterns and user behavior	E-commerce workload design
RW10	Product Page UX Best Practices (Baymard Institute, 2026)	Report	Product browsing behaviourand interaction patterns	E-commerce read operation patterns
RW11	2024 Product Finding Research (Baymard Institute, 2024)	Report	Search and filtering behaviourin e-commerce	E-commerce query workload design

CHAPTER 3
METHODOLOGY

3.1 Overview

This project compares the performance of three cloud-hosted SQL databases using the same e-commerce workload. The goal is to observe how each database behaves under a realistic but controlled online shopping scenario. To keep the comparison fair, the same schema, the same synthetic dataset, the same indexes, and the same query structure are used for all three databases [30].

The study uses a **free-tier-only setup**. The databases were not normalized to have identical CPU or RAM resources [31] , because the intention was to test the databases in the way they are naturally offered to users on their free tiers. This makes the study practical and closer to how a small project or startup might actually use them.

The benchmark is based on a simplified e-commerce system because e-commerce naturally contains both read-heavy and write-heavy actions. Users search for products, browse categories, inspect product details, add items to a cart, update the cart, and complete purchases. These actions are useful for performance testing because they cover a wide range of database behaviour, from simple reads to transactional writes [24].

3.2 Database Selection

Three SQL databases were selected for the comparison:

- **Supabase**
- **Neon**

- **CockroachDB**

These databases were chosen because they are cloud-hosted, easy to access, and available on free tiers. They also represent slightly different deployment styles, which makes the comparison more meaningful. Since the focus of the project is on practical benchmarking, the free-tier environment was used as-is instead of trying to equalize hardware resources across providers [24].

The comparison is therefore not about forcing all systems into the same machine configuration. Instead, it measures how they perform under their natural free-tier conditions.

3.3 Schema Design

The benchmark uses a minimal but realistic **six-table e-commerce schema**. The schema is intentionally kept small so that the benchmark stays focused on database performance rather than unnecessary application complexity.

The schema was chosen because it reflects the basic structure of an online shopping system. The tables cover product browsing, cart management, checkout, and order tracking. The structure is also simple enough to be loaded and queried consistently across all databases [24], [29].

The schema follows common e-commerce design ideas and is also inspired by benchmark-style retail modelling such as TPC-DS, where the dataset structure is built around a typical business process rather than an artificial toy example [29].

3.4 Dataset Design and Size

The dataset is synthetic, but it is designed to look like a real e-commerce dataset. The same generated dataset is used across all three databases so that the comparison remains fair [25]. The Schema used is shown in Table 2.

The overall dataset size is kept roughly in the range of **80–150 MB**, which is appropriate for free-tier cloud databases. The goal is not to create an enterprise-scale warehouse, but to generate enough data to test query behaviour, index use, and write overhead without exceeding small cloud limits.

Table 2: E-commerce Schema and Dataset Specification

Table	Columns	Purpose	Row Count	Notes
categories	category_id, name, parent_id	Stores product categories and supports a simple two-level hierarchy	50	Includes flat and two-level nested categories
products	product_id, name, category_id, price, stock_qty, description, created_at	Stores the product catalog and inventory-related data	10,000	Main catalog table
users	user_id, email, name, created_at	Stores shopper information	5,000	Simulated shoppers
cart_items	cart_item_id, user_id, product_id, quantity, added_at	Stores items currently added to a cart	8,000	Roughly 1–3 active cart items per user

orders	order_id, user_id, status, total_amount, created_at	Stores order headers and order state	3,000	Historical orders
order_items	order_item_id, order_id, product_id, quantity, unit_price	Stores the individual items included in each order	9,000	Around 3 items per order on average

Dataset realism

The dataset was built to feel realistic rather than random. It includes:

- realistic product names,
- realistic user names and emails,
- realistic prices and stock values,
- timestamps,
- order statuses,
- and cart relationships.

A small amount of skew is also included. About **20% of products** are placed into "hot" categories, and about **20% of users** behave as active users. This creates a more realistic access pattern because real e-commerce traffic is not perfectly uniform. Some products and categories are naturally accessed far more often than others [24], [26].

The same dataset is reused unchanged across Supabase, Neon, and CockroachDB, which ensures that any performance difference comes from the database system and not from different input data.

3.5 Data Generation and Loading

The dataset is generated synthetically using **Python and Faker**. Faker is used to create realistic names, emails, text descriptions, and timestamps. Additional helper logic is used to keep the data consistent and to make sure the foreign keys match correctly across tables [27].

The data is loaded in a strict order so that dependencies are respected:

categories → products → users → cart_items → orders → order_items

This loading order is important because each table depends on earlier tables through foreign keys.

Before loading, the generated data is checked to make sure:

- foreign keys are valid,
- prices and quantities are realistic,
- stock quantities are not negative,
- cart items have unique user-product pairs,
- and order totals match the sum of their line items.

This makes the dataset usable directly without manual cleanup.

3.6 Indexing Strategy

Only a minimal set of indexes is used as shown in the Table 3. This keeps the benchmark realistic and avoids over-optimizing one database compared to the others.

Table 3: Indexing Strategy and Purpose

Index	Purpose
products(category_id)	Speeds up category browsing
products(name)	Speeds up product search
products(price)	Speeds up price-range filtering
cart_items(user_id)	Speeds up cart lookup by user
orders(user_id)	Speeds up order lookup by user

No extra indexes are added. This is intentional because the project also wants to measure the natural cost of writes such as cart updates and checkout operations. If too many indexes are added, write operations may become unrealistically expensive and distort the comparison.

3.7 Workload Design

Table 4 shows the workload distribution based on common e-commerce user actions divided into three tiers:

Table 4: Workload Tier Distribution

Tier	Operations	Share of Total Requests
Tier 1 – Discovery (read-heavy)	Search, browse category, product detail view	**93%**
Tier 2 – Cart (small write transactions)	Add to cart, update/remove cart	**5%**

Tier 3 – Purchase (large write transactions)	Checkout, payment completion	**2%**

This structure reflects how e-commerce systems are typically used. Most user activity is related to discovering products, while cart and checkout actions happen less often but are more important for conversion [28], [32].

Total workload size

Each test run contains **10,000 total operations**.

Table 5: Operation-wise Workload Breakdown (10,000 Queries)

Operation	Share	Query Count
Search	25%	2,500
Browse Category	30%	3,000
Product Detail View	38%	3,800
Add to Cart	3.5%	350
Update / Remove Cart	1.5%	150
Checkout	1.2%	120
Payment Completion	0.8%	80

This query structure shown in Table 5 gives a **93/7 read-write ratio**, which is suitable for e-commerce because browsing and searching dominate the user journey, while writing operations happen less frequently but remain important.

Why this mix is useful

- **Search, browse, and product detail** represent the bulk of user-facing reads.
- **Cart actions** represent small but important writes.
- **Checkout and payment** represent the most critical transactional operations.

This makes the workload useful for measuring both fast reads and slower, more sensitive write transactions [32], [33].

3.8 Query Structure

Each operation is represented by a query pattern that reflects a real e-commerce action.

Search

Search queries read product data, category data, and use pagination. They represent keyword search behaviour on the storefront.

Browse category

Browse queries return products in a category with filters such as price range and stock availability.

Product detail view

Product detail uses a fan-out read pattern. It fetches the main product record and supporting data such as category information and the number of items sold.

Add to cart

This operation first checks stock and then inserts or updates the cart record.

Update / remove cart

These operations change or delete items already stored in the cart.

Checkout

Checkout is treated as a multi-statement transaction. It creates an order, copies cart items into order items, updates inventory, and clears the cart.

Payment completion

Payment completion updates the order status and uses an idempotency guard to avoid double submission.

This query structure is important because it makes the workload look like a real online shopping funnel rather than a random set of database calls.

3.9 Execution Strategy

The benchmark is run in a simple sequential manner.

Warm-up phase

Before the measured run, a warm-up phase is executed using **500 mixed read queries**. This is not included in the final metrics. The purpose of warm-up is to stabilize the database and allow caches and internal controllers to settle before performance measurement begins.

Main run

After warm-up, the full workload of **10,000 operations** is executed.

Iterations

The full workload is repeated **three times**.

- Warm-up: 1 pass
- Measured workload: 3 passes

The benchmark does **not** repeat each query individually. Each full 10,000-operation run is treated as one complete iteration. This keeps the benchmark simple and avoids unnecessary complexity.

Randomized order

Although the number of queries per operation is fixed, the order of execution is randomized in each run. This is important because it prevents the benchmark from following a fixed pattern and makes the workload more realistic. It also reduces the chance that results are affected by a predictable sequence.

No concurrency

The benchmark uses **single-query sequential execution only**. This makes the comparison easier to understand and ensures that the measured results reflect the basic performance of the database rather than the behaviour of a connection pool or concurrent scheduler.

Fairness rules

To keep the comparison fair, the following remain the same across all databases:

- same schema,
- same dataset,
- same indexes,
- same query text,
- same client region,
- same general execution plan.

3.10 Performance Metrics

Table 6 summarises five performance metrics for each database that the benchmark records.

Table 6: Performance Metrics Definition and Collection Method

Metric	Meaning	Why it matters
Average latency	The average time taken to complete a query	Shows overall speed
P95 latency	Latency experienced by the slower 5% of requests	Shows common tail behavior
P99 latency	Latency experienced by the slowest 1% of requests	Shows worst-case user experience
Throughput	Number of queries completed per second	Shows how much work the database can handle
Error rate	Percentage of queries that fail or time out	Shows reliability and stability

Average latency is useful, but it does not tell the whole story. P95 and P99 are especially important because user experience is often affected by slow outliers, not just the average case. Throughput shows how much load the database can sustain, and error rate shows whether the system remains reliable under the same workload [34].

3.11 Summary of the Methodology

This methodology uses a simplified but realistic e-commerce schema, a synthetic dataset inspired by real retail behaviour, a minimal set of indexes, and a fixed workload split of **93% discovery**, **5% cart**, and **2% purchase**. The same dataset and query structure are applied to all three cloud SQL databases, and the benchmark is run sequentially with one warm-up pass and three measured passes.

The main purpose of this design is to compare how the databases behave under the same practical shopping workload while keeping the experiment simple, reproducible, and easy to explain.

CHAPTER 4
RESULTS

The benchmark was run with one warm-up pass and three measured iterations for each database. The tables below show the **average of the three measured iterations** for each operation. The figures referenced in the text should later be generated from these same averaged values.

4.1 Supabase Results

Table 7: Supabase — averaged results across 3 iterations

Operation	Avg Latency (ms)	P95 (ms)	P99 (ms)	Throughput (QPS)	Error Rate (%)
Search	98.14	105.56	134.60	1.76	0.00
Browse Category	94.72	101.80	125.27	2.11	0.00
Product Detail	188.65	198.58	248.08	2.67	0.00
Add to Cart	181.75	199.49	252.62	0.25	6.57
Update / Remove Cart	93.73	111.47	148.86	0.11	0.00
Checkout	721.46	1283.02	1791.12	0.08	14.44
Payment Completion	96.19	115.43	366.76	0.06	0.00
Overall	**141.83**	**197.07**	**517.07**	**7.02**	**0.40**

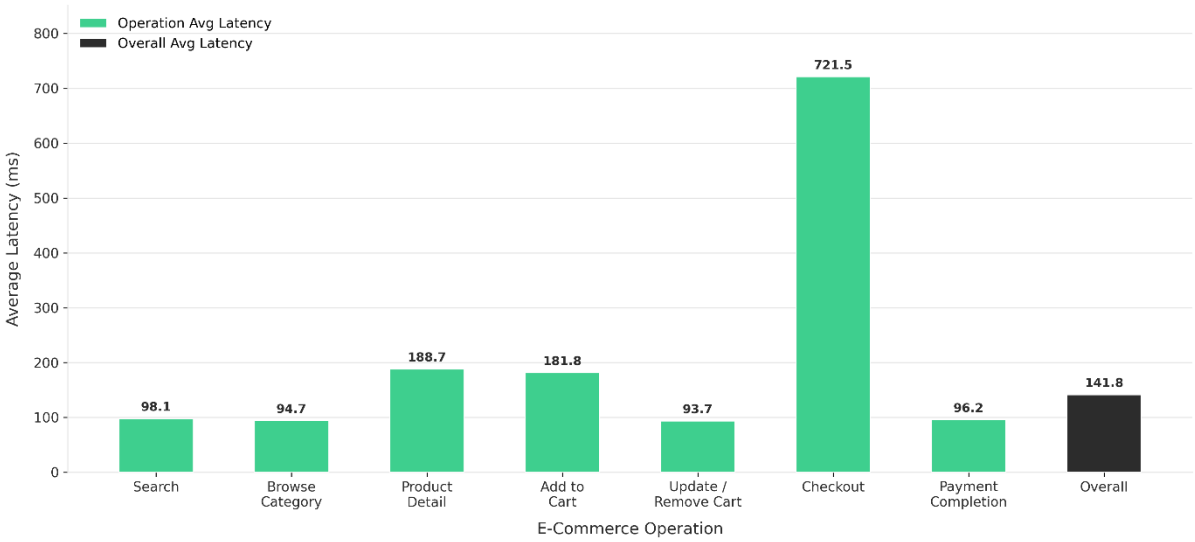

Fig. 1 Operation-wise Average Latency for Supabase

Table 7 and Fig. 1 clearly show the mixed performance profile of Supabase. The read-heavy operations were fairly stable, with search, browse category, and product detail staying in the same general range. Browse category was the fastest read operation, while product detail was naturally slower because it involved a wider read pattern. Cart-related updates were also manageable, but the add-to-cart path was noticeably slower than the pure reads because it combined a read check with a write update.

The main bottleneck for Supabase was **checkout**. It was the slowest operation by a large margin and also the one with the highest error rate. This is important because checkout is the most critical purchase-stage action in the workload. Payment completion was much lighter than checkout, but it still showed tail behaviour above the average. Overall, Supabase performed reasonably well on the discovery side of the workload but struggled more on the transactional purchase side.

4.2 CockroachDB Results

Table 8: CockroachDB — averaged results across 3 iterations

Operation	Avg Latency (ms)	P95 (ms)	P99 (ms)	Throughput (QPS)	Error Rate (%)
Search	97.85	115.67	345.05	3.04	0.00
Browse Category	53.15	83.42	108.24	3.66	0.00
Product Detail	87.21	91.86	155.64	4.63	0.00
Add to Cart	87.73	94.05	183.91	0.43	7.33
Update / Remove Cart	41.30	42.89	62.27	0.18	0.00
Checkout	333.00	580.66	820.64	0.15	13.61
Payment Completion	42.05	44.24	117.20	0.10	0.00
Overall	**81.57**	**106.42**	**347.43**	**12.18**	**0.42**

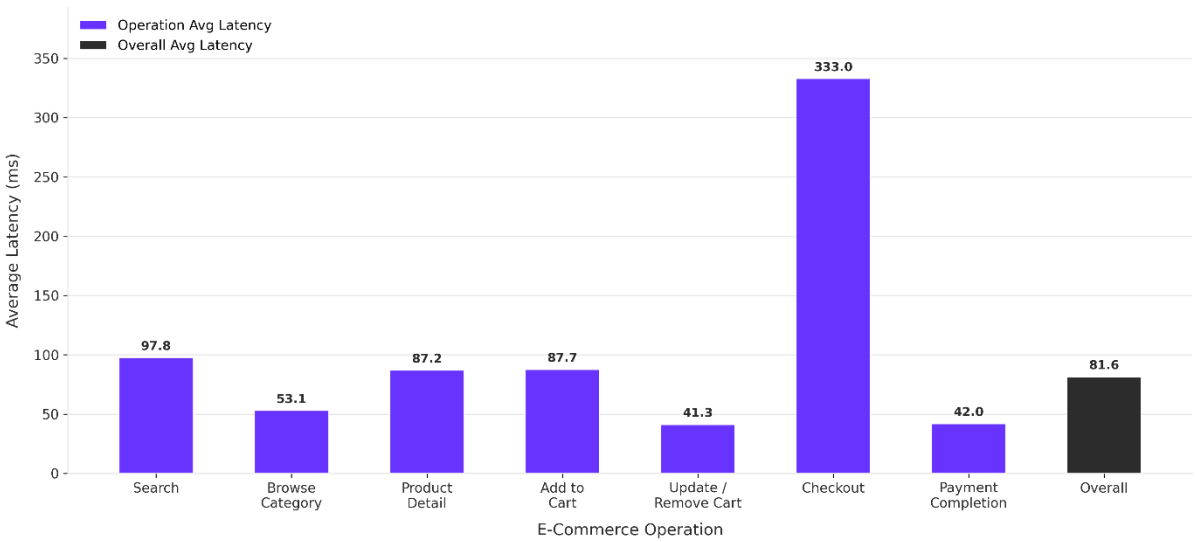

Fig. 2 Operation-wise Average Latency for CockroachDB

CockroachDB was the strongest overall performer in this benchmark. It had the lowest overall average latency, the highest throughput, and the best read-side performance across the majority of operations as shown in Table 8 and Fig. 2. Browse category, product detail, update/remove cart, and payment completion were all handled particularly well. These operations are important because they represent the most common shopping actions and the smaller write transactions that happen before checkout.

Checkout was still the hardest operation for CockroachDB, as expected, because it involved a multi-statement transaction touching several tables. Even so, its checkout latency was significantly lower than the corresponding checkout times seen in Supabase and Neon. The tail behaviour also stayed more controlled than in the other systems, although search still showed a relatively high P99 compared with its average. That suggests that search latency was generally good, but a small number of slower search requests still occurred.

Overall, CockroachDB offered the best balance between discovery queries and transactional writes in this workload.

4.3 Neon Results

Table 9: Neon — averaged results across 3 iterations

Operation	Avg Latency (ms)	P95 (ms)	P99 (ms)	Throughput (QPS)	Error Rate (%)
Search	103.33	125.47	189.86	1.65	0.00
Browse Category	101.84	114.93	140.41	1.99	0.00
Product Detail	201.22	209.20	261.17	2.51	0.00
Add to Cart	196.12	210.44	276.00	0.23	5.62
Update / Remove Cart	101.24	105.20	243.09	0.10	0.00
Checkout	770.69	1310.04	2334.43	0.08	12.22
Payment Completion	103.54	120.04	359.18	0.06	0.00
Overall	**151.31**	**207.73**	**506.89**	**6.61**	**0.34**

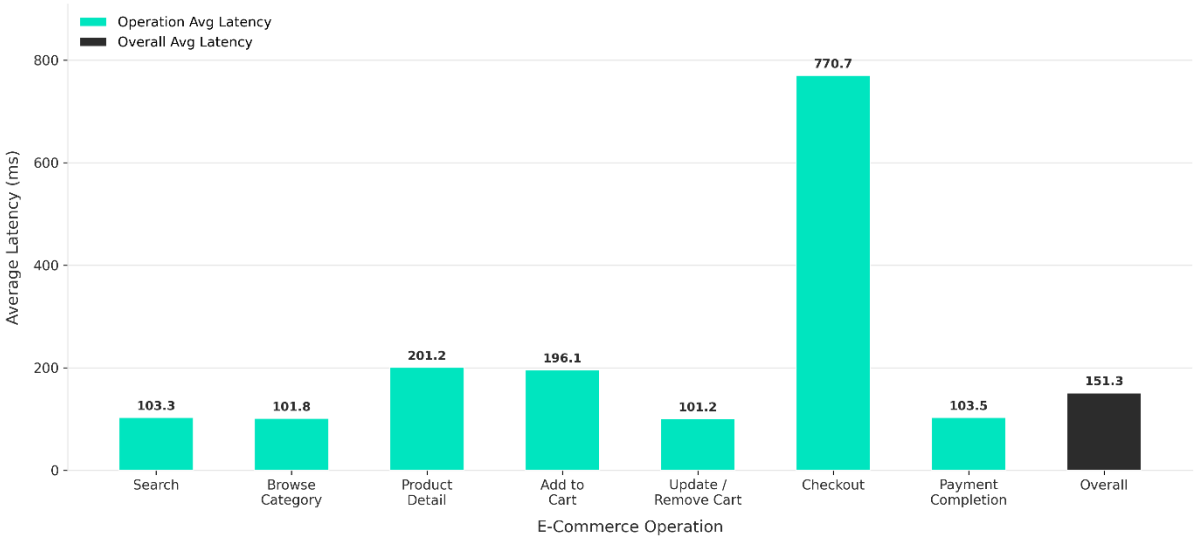

Fig. 3 Operation-wise Average Latency for Neon

Neon was the slowest of the three systems overall and it's performance is summarised in Table 9 and Fig. 3. Its discovery operations were consistently slower than CockroachDB and usually slower than Supabase as well. Product detail and add-to-cart were especially heavy, which matters because those are the points where a user moves from browsing into a more committed shopping state. In the workload, those operations happen frequently enough to affect the overall experience noticeably.

Checkout was again the biggest bottleneck. Neon showed the highest average checkout latency and the highest P99 in that operation group. That means the database could complete the workload, but the transactional part of the shopping flow was much more expensive than the read-heavy discovery phase. The overall throughput was also the lowest of the three databases.

Neon's error rate stayed low overall, which means most queries still completed successfully, but its speed profile was weaker across nearly every operation type.

4.4 Cross-Comparison of the Three Databases

Table 10: Final combined summary across all three databases

Database	Avg Latency (ms)	P95 (ms)	P99 (ms)	Throughput (QPS)	Error Rate (%)
Supabase	141.83	197.07	517.07	7.02	0.40
CockroachDB	81.57	106.42	347.43	12.18	0.42
Neon	151.31	207.73	506.89	6.61	0.34

The final comparison in Table 10 shows a clear ranking. **CockroachDB performed best overall**, with the lowest average latency and the highest throughput. **Supabase** came second, while **Neon** was the slowest overall. The difference is most visible in the average latency and throughput metrics, which makes the overall performance gap easy to interpret.

In relative terms, CockroachDB reduced average latency by about **42.5%** compared with Supabase and about **46.1%** compared with Neon. It also delivered much higher throughput, which is especially important for a workload dominated by discovery operations. The discovery tier is where most of the total requests occur, so a database that handles reads efficiently has a major advantage.

The checkout stage was the most difficult part of the workload for all three systems. This is expected because checkout is a multi-table transactional operation and includes inventory updates and cart cleanup. Even though all three databases completed the workload, the tail latency for checkout was much higher than for the discovery operations. That difference confirms

that the workload successfully separates simple read-heavy behaviour from more expensive transaction-heavy behaviour.

Another useful observation is that low error rate alone does not guarantee strong performance. Neon had a slightly lower overall error rate than the other two, but its latency and throughput were still weaker. This shows why the benchmark needs to report both speed and reliability metrics together rather than relying on only one measurement.

4.5 Short concluding summary for the Results section

The benchmark showed that the three cloud SQL databases behaved differently under the same e-commerce workload. CockroachDB gave the best overall latency and throughput, Supabase was in the middle, and Neon was the slowest. Read-heavy operations such as search, browse, and product detail were generally faster than checkout and other write-heavy actions, which matches the structure of a realistic e-commerce application. The figures and tables together should make this comparison easy to read and easy to defend.

CHAPTER 5
FUTURE SCOPE

This project can be extended in several useful directions in the future. The current work focuses on a simple and practical comparison of cloud-hosted SQL databases using a controlled e-commerce workload. In the future, this can be developed into a more complete benchmarking framework for startups and small businesses that need to choose the right database for e-commerce applications.

One possible future improvement is to build a more advanced benchmark that includes a wider range of real-world e-commerce scenarios. This could involve more complex shopping behaviour, larger datasets, more detailed product and order structures, and additional performance indicators such as cost efficiency, scalability, and long-term stability. Such an expanded benchmark would help provide a more complete view of how different databases behave under different business conditions.

Another possible extension is to include concurrency and user simulation. In the current project, the workload is executed sequentially in order to keep the comparison simple and easy to understand. In the future, a separate tool could be developed to generate concurrent load based on user requirements. This would allow the benchmark to simulate multiple users shopping at the same time, which would make the evaluation more realistic for active online stores and startup platforms.

The benchmark could also be expanded to support a more flexible testing environment, where the user can choose the number of simulated users, the workload intensity, and the type of e-commerce actions to run. This would make the tool more adaptable for different business sizes, from early-stage startups to growing online stores.

Overall, the future scope of this work is to move from a simple comparison study to a full benchmarking system for e-commerce database selection, with support for realistic workload generation, concurrent execution, and broader performance evaluation.

REFERENCES

1. India Brand Equity Foundation (IBEF), "E-Commerce Industry in India," Ministry of Commerce and Industry, Government of India, 2025. [Online]. Available: https://www.ibef.org/industry/ecommerce
2. Press Information Bureau, Government of India, "India's Startup Revolution," Press Release, Dec. 31, 2024. [Online]. Available: https://pib.gov.in/PressReleasePage.aspx?PRID=2098452
3. Supabase, Inc., "Supabase Documentation — Database," 2025. [Online]. Available: https://supabase.com/docs/guides/database/overview
4. MongoDB, Inc., "MongoDB Atlas Documentation," 2025. [Online]. Available: https://www.mongodb.com/docs/atlas
5. Cockroach Labs, Inc., "CockroachDB Documentation," 2025. [Online]. Available: https://www.cockroachlabs.com/docs
6. "Comparative study of SQL vs. NoSQL for high-performance e-commerce databases," *International Journal of Advanced Research*, ResearchGate, 2024. [Online]. Available: https://www.researchgate.net/publication/384227409
7. "Backend databases: Comparing SQL vs. NoSQL in 2024," Slashdev.io, 2024. [Online]. Available: https://slashdev.io/backend-databases-comparing-sql-vs-nosql-in-2024
8. D. Van Landuyt, M. Levrau, V. Reniers, and W. Joosen, "An e-commerce benchmark for evaluating performance trade-offs in document stores," *Proc. Companion of the 2019 ACM/SPEC International Conference on Performance Engineering (ICPE)*, 2019, pp. 53–58.
9. H. W. Cain, R. Rajwar, M. Marden, and M. H. Lipasti, "An architectural evaluation of Java TPC-W," *Proc. 7th International Symposium on High-Performance Computer Architecture (HPCA)*, 2001, pp. 229–240.
10. J. S. van der Veen, B. van der Waaij, and R. J. Meijer, "Sensor data storage performance: SQL or NoSQL, physical or virtual," *Proc. IEEE 5th International Conference on Cloud Computing*, 2012, pp. 431–438.
11. P. Grzesik and D. Mrozek, "Comparative analysis of time series databases in the context of edge computing for low power sensor networks," *Proc. IFIP International Conference on Computer Information Systems and Industrial Management*, 2020, pp. 371–383.

12. Y. Li and S. Manoharan, "A performance comparison of SQL and NoSQL databases," *Proc. IEEE Pacific Rim Conference on Communications, Computers and Signal Processing*, 2013, pp. 15–19.
13. V. Abramova and J. Bernardino, "NoSQL databases: MongoDB vs Cassandra," *Proc. International C* Conference on Computer Science and Information Systems*, 2013, pp. 14–22.
14. A. Makris, K. Tserpes, G. Spiliopoulos, and D. Anagnostopoulos, "Performance evaluation of MongoDB and PostgreSQL for spatio-temporal data," *Proc. EDBT/ICDT Workshops*, 2019.
15. J. Klein, I. Gorton, N. Ernst, P. Donohoe, K. Pham, and C. Matser, "Performance evaluation of NoSQL databases: A case study," *Proc. 1st Workshop on Performance Analysis of Big Data Systems*, 2015, pp. 5–10.
16. F. R. Oliveira and L. del Val Cura, "Performance evaluation of NoSQL multi-model data stores in polyglot persistence applications," *Proc. International Conference on Model and Data Engineering*, 2017, pp. 230–241.
17. W. W. Moe, "Buying, searching, or browsing: Differentiating between online shoppers using in-store navigational clickstream," *Journal of Consumer Psychology*, vol. 13, no. 1–2, pp. 29–39, 2003.
18. J. B. Kim, P. Albuquerque, and B. J. Bronnenberg, "Mapping online consumer search," *Journal of Marketing Research*, vol. 48, no. 1, pp. 13–27, Feb. 2011.
19. S. C. Necula, "Exploring the impact of time spent reading product information on e-commerce websites: A machine learning approach to analyse consumer behaviour," *Electronics*, vol. 12, no. 9, 2023, Art. no. 2073.
20. J. Hwang, "E-commerce and consumer behaviour," *International Journal of Advanced Research*, vol. 11, no. 5, pp. 456–472, 2023.
21. Baymard Institute, "The current state of checkout UX," 2023–2025. [Online]. Available: https://baymard.com/checkout-usability
22. Baymard Institute, "E-commerce product finding: 2024 expanded research findings," 2024. [Online]. Available: https://baymard.com/product-finding
23. Baymard Institute, "Product page UX 2026: Pitfalls and best practices," 2023–2026. [Online]. Available: https://baymard.com/product-page
24. D. Van Landuyt, M. Levrau, V. Reniers, and W. Joosen, "An e-commerce benchmark for evaluating performance trade-offs in document stores," in *Proc. DaWaK 2024 – Big Data*

Analytics and Knowledge Discovery, Lecture Notes in Computer Science, vol. 14912, Springer, 2024, pp. 239–253.

25. "Advancing retail data science: Comprehensive evaluation of synthetic data generation frameworks," *arXiv preprint arXiv:2406.13130*, June 2024.
26. A. Berke, D. Calacci, R. Mahari, T. Yabe, K. Larson, and A. Pentland, "Open e-commerce 1.0: Five years of crowdsourced U.S. Amazon purchase histories with user demographics," *Scientific Data*, vol. 11, 2024, Art. no. 491. DOI: 10.1038/s41597-024-03329-6.
27. "An e-commerce dataset revealing variations during sales promotions," in *Proc. 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2024)*, 2024, pp. 3102–3112. DOI: 10.1145/3626772.3657870.
28. P. Normann, S. Baumeister, and T. Wilm, "OTTO recommender systems dataset," Kaggle, 2023. DOI: 10.34740/KAGGLE/DSV/4991874. [Online]. Available: https://github.com/otto-de/recsys-dataset
29. Transaction Processing Performance Council, "TPC-DS benchmark specification, version 3.2.0," 2021. [Online]. Available: http://www.tpc.org/tpcds/
30. "What is database benchmarking?" Milvus AI Quick Reference, 2024. [Online]. Available: https://milvus.io/ai-quick-reference/what-is-database-benchmarking
31. "Database benchmarking best practices," BenchAnt Blog, 2024. [Online]. Available: https://benchant.com/blog
32. "Best practices for database benchmarking," Aerospike, 2024. [Online]. Available: https://aerospike.com/blog/best-practices-for-database-benchmarking/
33. "Benchmarking Postgres," PlanetScale Blog, 2024. [Online]. Available: https://planetscale.com/blog/benchmarking-postgres
34. "P50 vs P95 vs P99 latency percentiles: Understanding performance metrics," OneUptime Blog, 2025. [Online]. Available: https://oneuptime.com/blog/post/2025-09-15-p50-vs-p95-vs-p99-latency-percentiles/

PROOF OF SUBMISSION

Review Complete

Last review activity: 10th February 2026

Watch to learn what we're doing behind the scenes

Manuscript number:

FGCS-D-25-04508

Need more help?

Please visit our Journal Article Publishing Support Center